

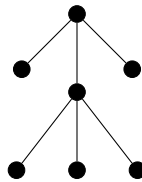
Taller de Lógica II: Bifurcaciones y Recursión

Monitorías Sistemas Operativos

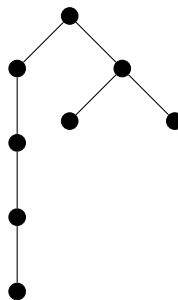
31 de agosto de 2026

Bloque 1: Diseño de jerarquías

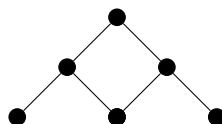
Ejercicio 1



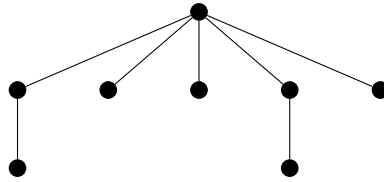
Ejercicio 2



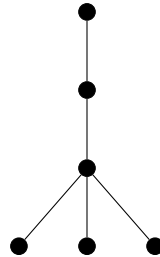
Ejercicio 3



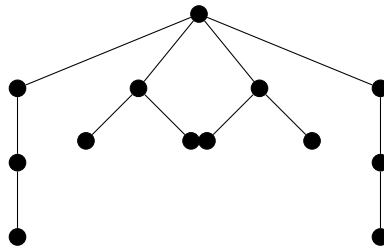
Ejercicio 4



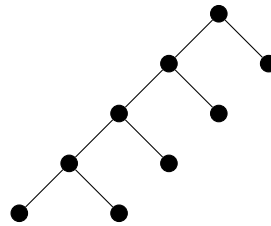
Ejercicio 5



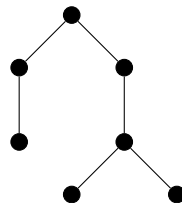
Ejercicio 6

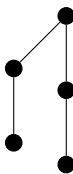
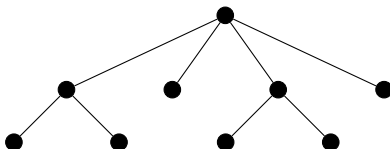


Ejercicio 7



Ejercicio 8



Ejercicio 9**Ejercicio 10****Bloque 2: Análisis de código**

Para cada caso, determine procesos totales, árbol y procesos que alcanzan el último printf.

Ejercicio 11

```
for (int i = 0; i < 3; i++)
    if (fork() == 0) break;
printf("X\n");
```

Ejercicio 12

```
if (fork() != 0) {
    fork();
} else if (fork() == 0) {
    fork();
}
```

Ejercicio 13

```
pid_t p = fork();
if (p == 0) {
    for (int i = 0; i < 2; i++) fork();
}
```

Ejercicio 14

```
if (fork() || fork()) {
    if (fork() == 0) printf("A\n");
}
```

Ejercicio 15

```
for (int i = 0; i < 2; i++) {  
    if (fork() != 0) break;  
}
```

Ejercicio 16

```
switch (fork()) {  
case 0: fork(); break;  
default: if (fork() == 0) fork();  
}
```

Ejercicio 17

```
pid_t a = fork();  
if (a == 0 && fork() != 0) fork();
```

Ejercicio 18

```
if (fork() == 0) {  
    fork();  
} else {  
    for (int i = 0; i < 2; i++) fork();  
}
```

Ejercicio 19

```
for (int i = 0; i < 2; i++) {  
    pid_t p = fork();  
    if (p == 0) break;  
}
```

Ejercicio 20

```
void crear(int n) {  
    if (n <= 0) return;  
    if (fork() == 0) crear(n - 1);  
    else if (n == 2) fork();  
}  
int main(void) { crear(3); }
```